

Министерство высшей науки и образования РФ

Политехнический институт

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2 по курсу

«Логика и основы алгоритмизации в инженерных задачах»

на тему «Оценка времени выполнения программ»

Выполнили: ст. гр. 25BBB2

Тургенев И.В.

Козлов А.В.

Проверил:

к.т.н. доцент Юрова О.В.

к.т.н. доцент Деев М.В.

Пенза 2026

Название

Оценка времени выполнения программ

Цель работы:

Оценить время выполнения программ языка Си или их частей. Они могут использоваться средствами, предоставляемые библиотекой `time.h`.

Лабораторное задание:

Задание 1:

1. Вычислить порядок сложности программы (O-символику);
2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000;
3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива;
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел;
3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел;
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую;

5. Оценить время работы стандартной функции qsort, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Описание использованных методов для решения заданий

Задание 1

1. Вычислить порядок сложности программы (О-символику);

Общее количество операций:

$$N \cdot N \cdot N = N^3$$

Следовательно, временная сложность алгоритма:

$$O(N^3)$$

Функция заполнения матрицы

```
void fill_matrix(int* m, int n)
{
    for (int i = 0; i < n * n; i++)
        m[i] = rand() % 100 + 1;
}
```

Назначение

Заполняет матрицу случайными числами.

Параметры

int* m

Указатель на начало массива.

int n

Размер квадратной матрицы.

Принцип работы

Матрица хранится как одномерный массив.

Количество элементов:

$n * n$

Каждый элемент получает случайное значение:

`rand() % 100 + 1`

то есть число от 1 до 100.

Основная функция программы

`int main()`

Является точкой входа в программу.

Массив размеров матриц

`int sizes[] = {100, 200, 400, 1000, 2000, 4000, 10000};`

Содержит размеры тестируемых матриц.

Будут исследоваться матрицы:

100×100

200×200

400×400

1000×1000

2000×2000

4000×4000

10000×10000

Цикл тестирования

`for (int s = 0; s < count_sizes; s++)`

Последовательно перебирает все размеры матриц.

Получение текущего размера

```
int N = sizes[s];
```

Например:

100

200

400

...

Начало измерения общего времени

```
program_start = clock();
```

Фиксируется момент начала работы программы для данного размера матрицы.

Выделение памяти

```
int* A = (int*)malloc(N * N * sizeof(int));
```

```
int* B = (int*)malloc(N * N * sizeof(int));
```

```
int* C = (int*)malloc(N * N * sizeof(int));
```

Создаются три матрицы:

A

Первая матрица.

B

Вторая матрица.

C

Матрица результата.

Проверка выделения памяти

```
if (A == NULL || B == NULL || C == NULL)
```

Проверяется успешность выделения памяти.

Если памяти недостаточно:

```
printf("Memory allocation error...");
```

выводится сообщение об ошибке.

Заполнение матриц

```
fill_matrix(A, N);
```

```
fill_matrix(B, N);
```

Матрицы A и B заполняются случайными значениями.

Начало измерения времени умножения

```
mult_start = clock();
```

Фиксируется момент начала вычисления произведения матриц.

Умножение матриц

Используется классический алгоритм.

```
for (int i = 0; i < N; i++)
```

Перебор строк первой матрицы.

```
for (int j = 0; j < N; j++)
```

Перебор столбцов второй матрицы.

```
int sum = 0;
```

Переменная для хранения суммы произведений элементов.

```
for (int r = 0; r < N; r++)
```

Перебор элементов строки и столбца.

Формула умножения

```
sum += A[i * N + r] * B[r * N + j];
```

Реализует математическую формулу:

$$C_{ij} = \sum_{r=1}^N A_{ir} \cdot B_{rj}$$

Запись результата

```
C[i * N + j] = sum;
```

Полученный элемент помещается в результирующую матрицу.

Завершение измерения умножения

```
mult_end = clock();
```

Фиксируется момент окончания вычислений.

Освобождение памяти

```
free(A);
```

```
free(B);
```

```
free(C);
```

Удаляются все выделенные массивы.

Это предотвращает утечку памяти.

Завершение измерения общего времени

```
program_end = clock();
```

Фиксируется момент окончания выполнения всех операций.

Вычисление общего времени программы

```
double program_time =
```

```
(double)(program_end - program_start)
```

```
/
```

```
CLOCKS_PER_SEC;
```

Показывает время:

- выделения памяти;
- заполнения матриц;
- умножения;
- освобождения памяти.

Вычисление времени умножения

```
double multiply_time =  
(double)(mult_end - mult_start)  
  
/  
  
CLOCKS_PER_SEC;
```

Показывает время только операции умножения матриц.

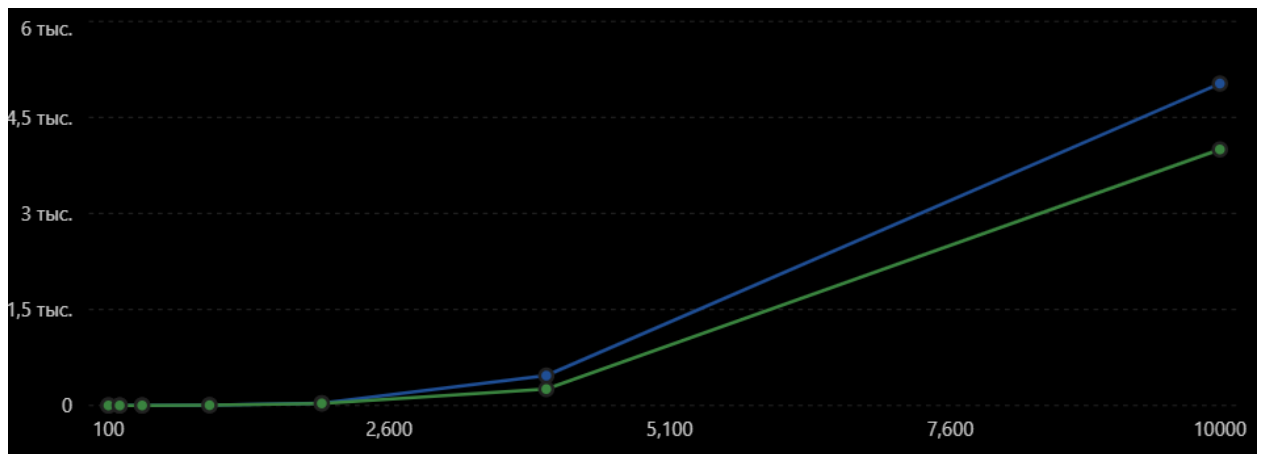
Вывод результатов

```
printf("%d\t%.6f\t%.6f\n",  
  
N,  
  
program_time,  
  
multiply_time);
```

Выводит строку таблицы:

```
100  0.003214  0.002781  
  
200  0.026542  0.025903  
  
400  0.214537  0.212106  
  
...
```

3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.



Ось ОУ- Время умножения(сек); Ось ОХ- Размер матрицы;

Зеленая линия-Теоретическая оценка; Синяя линия – Реальный запуск программы.

Задание 2

```
#define MAX_N 50000
```

Определяет максимальный размер массива.

Программа тестирует массивы до 50000 элементов, поэтому память выделяется заранее.

Алгоритм сортировки Шелла

```
void shell(int* items, int count)
```

Функция реализует сортировку Шелла.

Принцип работы

Сначала элементы сравниваются через большие промежутки.

Затем промежуток уменьшается.

В конце выполняется обычная сортировка вставками.

Последовательность шагов:

```
int a[5] = {9, 5, 3, 2, 1};
```

Работа алгоритма

Например:

9 5 7 1 4 8

Сначала сравниваются элементы через 9 позиций, затем через 5, потом через 3 и т.д.

Когда шаг становится равен 1, массив оказывается почти отсортированным и сортировка выполняется очень быстро.

Алгоритм Quick Sort

```
void qs(int* items, int left, int right)
```

Реализует рекурсивную быструю сортировку.

Выбор опорного элемента

```
int pivot = items[(left + right) / 2];
```

Опорный элемент выбирается из середины массива.

Например:

8 4 1 6 3 7 2

Середина:

6

Разбиение массива

Используются два указателя:

```
i = left;
```

```
j = right;
```

Левый движется вправо:

```
while(items[i] < pivot)
```

```
i++;
```

Правый движется влево:

```
while(items[j] > pivot)
```

```
j--;
```

Когда находятся элементы не на своих местах, они меняются местами.

```
temp = items[i];
```

```
items[i] = items[j];
```

```
items[j] = temp;
```

Рекурсивная обработка

После деления массив делится на две части:

```
if(left < j)
```

```
    qs(items, left, j);
```

```
if(i < right)
```

```
    qs(items, i, right);
```

Левая и правая части сортируются отдельно.

Функция сравнения для qsort

```
int compare(const void* a, const void* b)
```

Используется библиотечной функцией:

```
qsort()
```

Логика сравнения

```
if(x > y) return 1;
```

```
if(x < y) return -1;
```

```
return 0;
```

Возвращает:

- положительное число — первый элемент больше второго;
- отрицательное число — первый элемент меньше второго;
- 0 — элементы равны.

Формирование тестовых массивов

Случайный массив

`fill_random()`

Заполняет массив случайными числами.

```
arr[i] = rand() % 1000;
```

Пример:

15 823 41 77 502

Возрастающий массив

`fill_ascending()`

Создаёт уже отсортированный массив.

Пример:

1 2 3 4 5 6 7

Убывающий массив

`fill_descending()`

Создаёт массив в обратном порядке.

Пример:

7 6 5 4 3 2 1

Смешанный массив

`fill_half_half()`

Первая половина отсортирована по возрастанию.

Вторая половина отсортирована по убыванию.

Например:

1 2 3 4 5 5 4 3 2 1

Такой тип массива часто используется для анализа поведения алгоритмов на частично упорядоченных данных.

Копирование массива

`copy_array()`

Каждая сортировка должна работать с одинаковыми исходными данными.

Поэтому исходный массив копируется:

`source -> arr1`

`source -> arr2`

`source -> arr3`

где:

`arr1` — Shell Sort

`arr2` — Quick Sort

`arr3` — qsort

Измерение времени выполнения

Для каждой сортировки создаётся отдельная функция.

Shell Sort

`double test_shell(int arr[], int n)`

Измерение времени:

`start = clock();`

```
shell(arr, n);
```

```
end = clock();
```

Возвращается:

```
(end - start) / CLOCKS_PER_SEC
```

в секундах.

Quick Sort

```
double test_qs(int arr[], int n)
```

Аналогично измеряется время работы функции:

```
qs()
```

qsort

```
double test_qsort(int arr[], int n)
```

Измеряется время выполнения библиотечной функции:

```
qsort()
```

Основная функция

```
int main()
```

Инициализация генератора случайных чисел

```
srand((unsigned)time(NULL));
```

Позволяет получать разные случайные последовательности при каждом запуске программы.

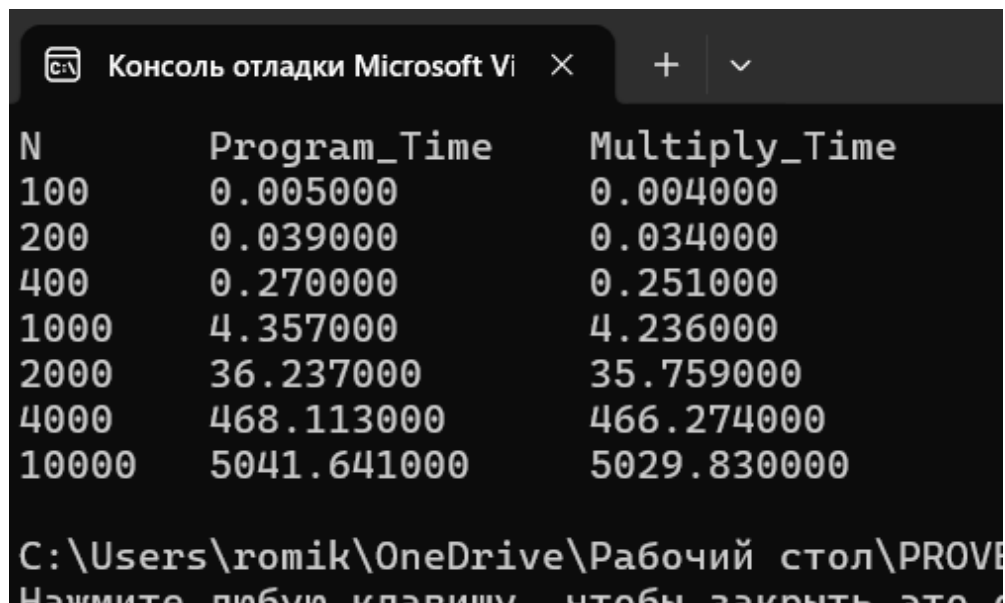
Размеры массивов

```
int sizes[]={1000,5000,10000,20000,50000};
```

На каждом размере проводится отдельное тестирование.

Результаты работы программы

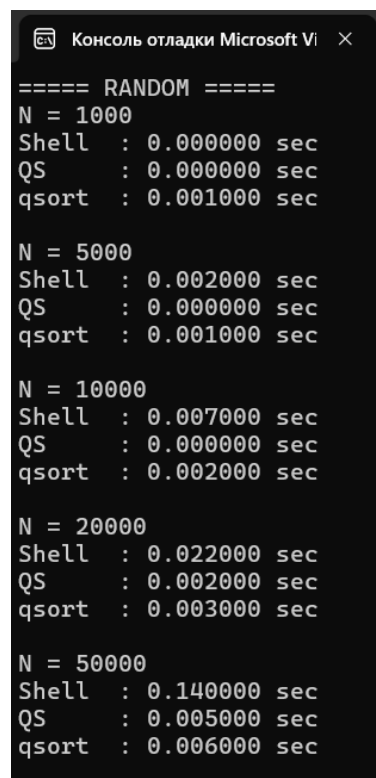
Задание 1:



N	Program_Time	Multiply_Time
100	0.005000	0.004000
200	0.039000	0.034000
400	0.270000	0.251000
1000	4.357000	4.236000
2000	36.237000	35.759000
4000	468.113000	466.274000
10000	5041.641000	5029.830000

C:\Users\romik\OneDrive\Рабочий стол\PROVE
Нажмите любую клавишу, чтобы закрыть это

Задание 2:



```
===== RANDOM =====
N = 1000
Shell : 0.000000 sec
QS    : 0.000000 sec
qsort : 0.001000 sec

N = 5000
Shell : 0.002000 sec
QS    : 0.000000 sec
qsort : 0.001000 sec

N = 10000
Shell : 0.007000 sec
QS    : 0.000000 sec
qsort : 0.002000 sec

N = 20000
Shell : 0.022000 sec
QS    : 0.002000 sec
qsort : 0.003000 sec

N = 50000
Shell : 0.140000 sec
QS    : 0.005000 sec
qsort : 0.006000 sec
```

```
===== ASCENDING =====  
N = 1000  
Shell : 0.000000 sec  
QS : 0.000000 sec  
qsort : 0.000000 sec  
  
N = 5000  
Shell : 0.000000 sec  
QS : 0.000000 sec  
qsort : 0.001000 sec  
  
N = 10000  
Shell : 0.000000 sec  
QS : 0.000000 sec  
qsort : 0.001000 sec  
  
N = 20000  
Shell : 0.000000 sec  
QS : 0.001000 sec  
qsort : 0.002000 sec  
  
N = 50000  
Shell : 0.000000 sec  
QS : 0.002000 sec  
qsort : 0.004000 sec
```

```
===== DESCENDING =====  
N = 1000  
Shell : 0.000000 sec  
QS : 0.000000 sec  
qsort : 0.000000 sec  
  
N = 5000  
Shell : 0.003000 sec  
QS : 0.000000 sec  
qsort : 0.000000 sec  
  
N = 10000  
Shell : 0.011000 sec  
QS : 0.000000 sec  
qsort : 0.001000 sec  
  
N = 20000  
Shell : 0.043000 sec  
QS : 0.001000 sec  
qsort : 0.002000 sec  
  
N = 50000  
Shell : 0.270000 sec  
QS : 0.001000 sec  
qsort : 0.004000 sec
```

```
===== HALF ASC / HALF DESC =====  
N = 1000  
Shell   : 0.000000 sec  
QS      : 0.000000 sec  
qsort   : 0.001000 sec  
  
N = 5000  
Shell   : 0.000000 sec  
QS      : 0.001000 sec  
qsort   : 0.000000 sec  
  
N = 10000  
Shell   : 0.003000 sec  
QS      : 0.000000 sec  
qsort   : 0.001000 sec  
  
N = 20000  
Shell   : 0.011000 sec  
QS      : 0.000000 sec  
qsort   : 0.001000 sec  
  
N = 50000  
Shell   : 0.073000 sec  
QS      : 0.001000 sec  
qsort   : 0.005000 sec
```

Вывод: В этой лабораторной работе мы поняли, как долго может компилироваться и выполняться проект на языке Си. Мы попробовали поработать с библиотекой «time.h». Компиляция нашего проекта вышла около полутора часа. А во втором задании удобнее и быстрее всего код работал с сортировкой QS, так как именно у этого вида сортировки было самое маленькое время среди остальных